OAU EEE: shared data and coding cheat sheet

Owner: whole team. Working proposal, 2 October 2026.

Use this alongside the separate API and ESP32 guide. Agree meanings before writing tables or functions.

What exists, and what this guide proposes

The repository has an initial scaffold and documentation. It has no implemented models, database schema or terminal API.

Retained names come from the handbook's teaching examples. New fields and interfaces here are proposals for team review, not claims about deployed code.

The source documents inform the design. The current request defines this deliverable. Institutional grading and approval rules remain decisions for the department.

The canonical working layout

Use the existing root-level app/, contracts/, simulators/, fixtures/, tests/, docs/ and deployment/ folders.

Write `app/students/models.py`, not `student/models.py`. Use plural students, academics, imports, accounts, reporting, identity and audit.

The walkthrough illustrated `software/app/`. The repository actually uses `app/`. Keep software/ reserved until the team agrees another purpose.

Where contributors begin

Area	First deliverable	Depends on
Pure Python	Validators, adapters, calculations and fictional fixtures	Agreed field meanings
Django	Models, services, scoped queries, forms and handlers	Dictionary and access rules
PostgreSQL	Constraint/index review and migration inspection	Reviewed Django models
Browser interface	Forms, permitted tables and error messages	View/form contracts
API and firmware	Shared JSON fixtures and two simulators	Stable student key and identity contract

Where contributors begin (continued)

Everyone starts with one synthetic student, one period and one course offering.

Existing names to retain

Retained handbook name	Meaning
`student_id`	UUID primary key of Student
`full_name`	`StudentProfile` display name
`source_system`, `identifier_value`	Source-scoped external identifier
`phone_number`, `admission_year`	Contact text and recorded year
score, `period_id`, `source_reference`	Numeric result, period key, attempt provenance
`is_active`, `occurred_at`	Operational state and aware event time
`credential_ref`, `diagnostic_code`	Device-scoped credential and controlled explanation

`student_name` is a local teaching variable. `full_name` is the shared profile field.

Your examples translated into shared names

Your description	Recommended canonical location/name
Student names	`StudentProfile.full_name`
Matriculation number	`ExternalIdentity.identifier_value` with `identifier_type="matric_number"`
Email and contact number	`StudentProfile.email_address` and `phone_number`
Admission year and mode	`StudentProfile.admission_year` and `admission_mode`
Guardian names and numbers	`GuardianContact.full_name`, `phone_number`, `email_address`
Current level and studentship	`AcademicStatusHistory.level` and `studentship_status`

Your examples translated into shared names (continued)

Your description	Recommended canonical location/name
Unique student identifier	`Student.student_id`, server-generated UUID

`identifier_type`, `email_address`, `admission_mode` and `AcademicStatusHistory` are proposed additions.

Your academic examples translated

Your description	Recommended canonical representation
GPA	`calculate_gpa` output gpa for an explicit period
CGPA	`calculate_cgpa` output cgpa through an explicit period
Courses for this semester	`list_registered_courses(student_id, period_id)`
All courses ever taken	`list_attempt_history(student_id)`
Courses available this semester	`CourseOffering` rows for `period_id`
Score	`ResultRevision`.score for one `RegistrationAttempt`
Leave years	Approved `LeaveOfAbsence` periods and a derived summary

Your academic examples translated (continued)

Offered, registered, attempted and passed mean different things. Preserve that distinction in names.

Important additions to your list

Add academic session, semester, programme, cohort, curriculum version, course code, title, units and course-version applicability.

Add attempt identity, registration state, result/review states, revision history, exemptions, substitutions and grading-policy version.

Add source ownership, file/row provenance, reviewer, timestamps, correction reasons, import conflicts and audit events.

Add device identity, credential mapping, expiring requests and minimal outcomes. Keep biometric templates outside academic records pending specialist design.

Type key for the dictionary

Code	Python	Django	PostgreSQL	JSON
U	UUID	UUIDField	uuid	UUID string
T(n)	str	`CharField(max_length=n)`	`varchar(n)`	string
I	int	`PositiveSmallIntegerField`	smallint plus CHECK	integer
D(p,s)	Decimal	`DecimalField(p,s)`	`numeric(p,s)`	decimal string
B	bool	`BooleanField`	boolean	boolean
DT	aware datetime	`DateTimeField`	timestamptz	ISO 8601 string
DATE	date	`DateField`	date	YYYY-MM-DD

Type key for the dictionary (continued)

FK means `ForeignKey` to the named entity. Its SQL type matches that entity's primary key.

Storage and absence conventions

All proposed new entity primary keys use UUIDs generated by the server. The handbook already uses `student_id` as Student's actual primary key.

Required means no NULL or blank. Optional text uses "" consistently. Optional numbers, dates and relationships use None / SQL NULL / JSON null.

Unknown state uses an explicit reviewed code. Missing score stays null, distinct from zero. A missing row is not a failed course.

Candidate lengths and decimal precision below are implementation proposals. Review real source formats before freezing them.

Student and profile dictionary

Entity.field	Type	Rule
`Student.student_id`	U	Required PK, uuid.uuid4, immutable
`Student.is_active`	B	Operational availability, distinct from studentship
`Student.created_at`	DT	Server creation time
`StudentProfile.student`	FK Student	One-to-one, required, protect history
`StudentProfile.full_name`	T(200)	Required trimmed text, never unique
`StudentProfile.email_address`	T(254), `EmailField`	Optional, validated, not assumed unique
`StudentProfile.phone_number`	T(32)	Optional text, preserve prefixes
`StudentProfile.admission_year`	I	Nullable until evidenced, credible reviewed range

Student and profile dictionary (continued)

Entity.field	Type	Rule
`StudentProfile.admission_mode`	T(32)	Controlled codes, including unknown

Optional split names require a real use case. Keep source spelling and name order.

External identity dictionary

Entity.field	Type	Rule
`ExternalIdentity.external_id entity_id`	U	Required PK
student	FK Student	Required
`source_system`	T(80)	Required source namespace
`identifier_type`	T(32)	Proposed discriminator, e.g. `matric_number`
`identifier_value`	T(100)	Required text, preserve zeros and punctuation
`is_active`	B	Current mapping state
`valid_from`, `valid_to`	DATE	Nullable historical bounds

External identity dictionary (continued)

The handbook's candidate unique key is (`source_system`, `identifier_value`). If multiple identifier types share a namespace, explicitly revise it to include `identifier_type`. No silent key change.

Guardian and relationship dictionary

Entity.field	Type	Rule
GuardianContact.guardian_id	U	Required PK
full_name	T(200)	Required for a recorded contact
phone_number, email_address	T(32), T(254)	Optional, at least one usable channel if policy requires
StudentGuardianLink.link_id	U	Required PK
student, guardian	FK Student, GuardianContact	Required links
relationship_type	T(32)	Reviewed code such as parent or guardian
contact_purpose	T(32)	Agreed purpose, e.g. emergency

Guardian and relationship dictionary (continued)

Entity.field	Type	Rule
`is_primary`	B	At most one primary per student/purpose if agreed

Use separate linked rows for multiple contacts. Never `guardian_phone_1`, `guardian_phone_2` or a comma-separated list.

Academic state and leave dictionary

Proposed entity.field	Type	Meaning
`AcademicStatusHistory.status_history_id`	U	History PK
student, period	FK Student, `AcademicPeriod`	Student's state in a specific period
`programme_code`, `cohort_reference`	T(32), T(40)	Reviewed programme and cohort labels
level	I, nullable	Recorded level, e.g. 200 in fictional fixtures
`studentship_status`	T(32)	active, `on_leave`, withdrawn, graduated, unknown: candidate codes
`curriculum_version`	FK `CurriculumVersion`, nullable	Applicable requirement set

Academic state and leave dictionary (continued)

Proposed entity.field	Type	Meaning
`LeaveOfAbsence.leave_id`, student	U, FK Student	Approved leave record
`start_period`, `end_period`	FK `AcademicPeriod`	Explicit bounds
`approval_reference`, reason	T(100), `TextField`	Evidence and restricted reason

Store approved leave. Flag gaps as suspected missing evidence. `current_level` is a query result for a selected period, not `admission_year` arithmetic.

Period and course dictionary

Entity.field	Type	Rule
`AcademicPeriod.period_id`	U	Required PK
`session_reference`, `semester_code`	T(32), T(16)	Required reviewed labels, unique pair
`starts_on`, `ends_on`	DATE	Optional until scheduled, ordered bounds
`Course.course_id`, `course_code`	U, T(32)	Stable key and reviewed code namespace
`CourseVersion.course_version_id`, course	U, FK Course	Version identity
`version_number`, title, `credit_units`	I, T(200), I	Required, units positive

Period and course dictionary (continued)

Entity.field	Type	Rule
`applicable_from`, `applicable_to`	FK `AcademicPeriod`, nullable	Explicit historical applicability
`CourseOffering.offering_id`	U	Required PK
`course_version`, period	FK `CourseVersion`, `AcademicPeriod`	Required exact offering

If sections exist, add `section_code` to the offering identity. Never rewrite old units when a new curriculum starts.

Curriculum and exception dictionary

Entity.field	Type	Meaning
`CurriculumVersion.curriculum_version_id`	U	Requirement-version PK
`programme_code, version_label`	T(32), T(40)	Required reviewed scope/version
`approval_reference`	T(100)	Authoritative policy reference
`CurriculumRule.rule_id, curriculum_version`	U, FK `CurriculumVersion`	Rule and parent
`requirement_type`	T(32)	Compulsory, elective-group etc.
`required_course, group_code`	FK Course nullable, T(32)	Rule-dependent shape
`minimum_units, minimum_courses`	I nullable	Rule-dependent reviewed thresholds

Curriculum and exception dictionary (continued)

Entity.field	Type	Meaning
`Exemption.exemption_id`, `Substitution.substitution_id`	U	Exception PKs
student, rule, `approval_reference`	FK Student, `CurriculumRule`, T(100)	Required authorisation

Substitution additionally references `replacement_course`. Validate rule shape and authority. Prefer explicit fields over arbitrary executable JSON rules.

Registration and result dictionary

Entity.field	Type	Rule
RegistrationAttempt.attempt_id	U	Required stable attempt PK
student, offering	FK Student, CourseOffering	Required
source_reference	T(100)	Source attempt reference, required when supplied
registration_status	T(32)	Candidate registered, withdrawn, unknown
ResultRevision.result_revision_id, attempt	U, FK RegistrationAttempt	Revision PK and parent
revision_number, score	I, D(5,2) nullable	Sequence and exact numeric result

Registration and result dictionary (continued)

Entity.field	Type	Rule
`grade_code`	T(8)	Optional until authoritative or calculated under policy
`source_state`, `review_state`	T(32)	Source approval distinct from department acceptance
supersedes	FK `ResultRevision` nullable	Earlier revision of the same attempt

D(5,2) supports fictional 0-100 values. Confirm official scale/precision. Accept zero, reject non-finite values and silent precision loss.

Result provenance and corrections

`ResultRevision` also needs `source_ref` T(100), reason `TextField`, `recorded_at` DT, `reviewed_at` nullable DT and `reviewed_by` nullable FK staff.

`append_result_revision` retains the previous revision and records why a change occurred. The selected effective revision must satisfy reviewed source and acceptance rules.

Do not store repeated uploads as new academic attempts. Do preserve a genuine later attempt of the same course.

Agree attempt uniqueness with the academic reviewer. A universal `unique(student, course)` would destroy legitimate repeats.

GPA and CGPA dictionary

Proposed output field	Type	Meaning
`student_id`, `period_id`	U	Calculation scope
gpa, cgpa	Decimal nullable, JSON decimal string	Derived under selected policy
`total_quality_points`	Decimal	Sum of counted `grade_point` * `credit_units`
`total_counted_units`	int or reviewed Decimal	Denominator
`grading_policy_id`	U	Exact policy version
`calculated_at`, `as_of`	DT	Run time and selected evidence boundary
`is_complete`, `diagnostic_code`	bool, str	Evidence completeness and explanation

GPA and CGPA dictionary (continued)

Derived output is not an independently editable Student field. Optional cached snapshots need policy/input-version provenance and invalidation.

Calculation rules that need agreement

GPA = `sum(counted grade_point * counted credit_units) / sum(counted credit_units)`. CGPA applies the reviewed cumulative counting policy.

Grade boundaries, rounding, failed/withdrawn courses, repeats, transfers, exemptions and corrected results need explicit rules. These are not established OAU rules in the sources.

Fictional example: grade point 4 for 3 units, grade point 3 for 2 units. GPA = 18/5 = 3.60. Do not average course grades or semester GPAs without weights.

Zero denominator returns gpa=None with a diagnostic. Incomplete data must carry an explicit completeness warning.

Grading policy and source ownership

Proposed `GradingPolicy`: `grading_policy_id` U, `policy_code` T(40), `version_number` I, `approval_reference` T(100), `effective_from`/`effective_to` FK period.

Proposed `GradeBand`: `band_id` U, policy FK, `grade_code` T(8), `minimum_score`/`maximum_score` Decimal, `grade_point` Decimal. Review inclusive/exclusive boundaries and reject overlaps.

Proposed `ProfileFieldProvenance`: `provenance_id` U, student FK, `field_name` T(80), `source_system` T(80), `source_reference` T(100), `recorded_at` DT, `changed_by` FK staff nullable.

Declare the owner of each field. An import must flag a conflict before overwriting a department-owned phone number.

Import dictionary

Entity.field	Type	Rule
`ImportBatch.batch_id`, `uploaded_by`	U, FK staff	Batch PK and authenticated uploader
`source_system`, `source_reference`	T(80), T(100)	Source identity
`file_ref`, `file_checksum`	T(255), T(64)	Private object key and SHA-256 text
period, status, `uploaded_at`	FK period nullable, T(32), DT	Scope, controlled state, time
`StagingRow.staging_row_id`, batch	U, FK `ImportBatch`	Row PK and parent
`row_number`, `raw_data`, `canonical_data`	`PositiveIntegerField`, JSONField, JSONField	Position and staged representations

Import dictionary (continued)

Entity.field	Type	Rule
`validation_errors`, `row_status`	JSONField, T(32)	Structured errors and review state
`ImportConflict.conflict_id`, `staging_row`	U, FK `StagingRow`	Conflicting evidence

JSONField maps to jsonb. Keep core academic entities relational. JSON does not excuse undefined field meanings.

Adapter names and import acceptance

Retain `external_identifier` and `period_reference` as handbook adapter keys. They are source lookup inputs, not Student model fields.

Map Matric No to `external_identifier`, Course to `course_code`, Score to score and Period to `period_reference`. Then resolve `source_system` + `identifier_value` to `student_id`.

Proposed structured error: `row_number`, `field_name`, `diagnostic_code`, message. Avoid raw exception traces or sensitive payloads in errors.

Acceptance records `reviewed_by`, `reviewed_at`, `accepted_at` and accepted references. Same bytes, same source record, conflicting revision and genuine repeat need different handling.

Accounts dictionary

Proposed record	Fields and meaning
Django staff user	Framework password hash, username/login and active state
`StaffScope`	`scope_id` U, user FK, `programme_code` T(32), `cohort_reference` T(40)
`RoleAssignment` if needed	`assignment_id` U, user FK, `role_code` T(40), `granted_by` FK, `granted_at` DT, `revoked_at` nullable DT
Permission vocabulary	`students.add_student` and reviewed custom `reporting.export_records`

Use Django users/groups/permissions first. Add scope records only where departmental access requires them. Select a custom user model before initial migrations if needed.

Accounts dictionary (continued)

A student record does not create a login. The proposed first release serves staff.

Reporting and protected file dictionary

`ExportJob`: `export_job_id` U, `requested_by` FK staff, `requested_at` DT, status T(32), selection JSONField, `snapshot_reference` T(100), `file_ref` T(255), `expires_at` nullable DT, `diagnostic_code` T(80).

Proposed `StudentDocument`: `document_id` U, student FK, `category_code` T(32), `reference_number` T(100), `revision_number` I, `file_ref` T(255), `content_type` T(100), `file_size_bytes` `PositiveBigIntegerField`, checksum T(64).

Also document `uploaded_by`, `uploaded_at` and supersedes. Confirm document categories, numbering, retention and scanning workflow before implementing uploads.

Store actual files privately. Database rows hold references and metadata. A `file_ref` is not an unrestricted public URL.

Audit and identity cross-reference

Retain `AuditEvent` fields actor, action, `target_type`, `target_id`, reason, `correlation_id` and `occurred_at`. Add `audit_event_id` UUID PK if using the shared new-key convention.

actor is a nullable staff FK only when system/device events require it. action and `target_type` are controlled text. `target_id` is text because target types may differ.

Device, `CredentialMapping`, `IdentityCheckRequest` and `IdentityEvent` live in `identity/`. Their field dictionary and authentication rules are in the separate API guide.

Never copy passwords, challenges, raw biometric material or whole result spreadsheets into audit messages.

Pure Python contributors

Write small functions whose inputs and outputs work without HTTP or a database.

`students/validation.py`: `normalise_name(value: str) -> str`. Reject wrong types and blank names, trim outer spaces, enforce agreed length.

`imports/adapters.py`: `adapt_row(source_row) -> CanonicalImportRow`. Preserve text identifiers and distinguish blank scores from zero.

`imports/validation.py`: `validate_row(row) -> list[RowError]`. Check shape and values. Services later resolve student/course existence and authority.

`academics/calculations.py`: `calculate_gpa(attempts, policy) -> GpaSummary`. Consume already selected, permitted evidence.

OOP choices

Django model classes represent persisted entities and relationships. Dataclasses such as `CanonicalImportRow`, `RowError` and `GpaSummary` describe typed values passed between functions.

Use a `SourceAdapter` interface when CSV and XLSX need different readers. Use a `GradingPolicy` strategy when reviewed policy versions differ. Dependency injection lets tests supply a fake reader or policy.

Keep `normalise_name`, `parse_score` and formatting as functions when they have no state. Avoid a class for every helper and a single Student class that handles imports, HTTP and calculations.

Composition lets an import service call adapters, validators and academic services without inheriting all their behaviour.

Pure Python acceptance checkpoints

Input type and missing-value tests must cover valid text, whitespace, leading zeros, zero scores, excessive precision, `NaN` and infinity.

Adapters expose row numbers and source provenance. Reader failures become clear errors. They never write directly to accepted academic tables.

Calculations test weighted units, corrected results, repeats under the chosen policy, missing evidence and a zero denominator.

Keep rule versions explicit. New source formats need a new adapter or mapping, not changes scattered through views.

Django file responsibilities

File under app/<module>/	Expected content
models.py	Model classes, relationships and database constraints
services.py	Authorised mutations, transactions and business operations
queries.py	Reusable reads with caller scope
forms.py	Browser input cleaning
views.py and urls.py	HTTP coordination and routes
`api/serializers.py`	JSON type/shape validation where an API exists
migrations/	Reviewed schema/data changes
tests/	Meaningful model, service and access checks

Django file responsibilities (continued)

Create files when they have work. `identity/services/` is a package, so do not also create `identity/services.py`.

app/students/models.py

Classes: Student, `StudentProfile`, `ExternalIdentity`, `GuardianContact`, `StudentGuardianLink`. Proposed supporting class: `ProfileFieldProvenance`.

Student owns `student_id`. `StudentProfile` has a `OneToOneField` named student with `related_name="profile"`. `ExternalIdentity` and guardian links refer to the stable key.

Services: `create_student(*, actor, full_name) -> Student`, `update_profile(*, actor, student_id, changes, reason)`, `link_guardian(*, actor, student_id, guardian_id, relationship_type, contact_purpose)`.

Queries: `get_student_for_view(*, user, student_id)`, `resolve_external_identity(*, source_system, identifier_value)`.

Students module checkpoints

A blank name and denied caller create no rows. Student, profile and required audit write commit together or roll back together.

Changing phone/name/matriculation mapping preserves `student_id`. A duplicate name does not merge people. Ambiguous external identity becomes review work.

Two students can share a guardian without sharing a `StudentProfile`. Protect referenced students from accidental deletion.

Retain the handbook's permission-aware retrieval pattern. Check both direct service calls and HTTP routes.

app/academics/models.py

Classes: `AcademicPeriod`, Course, `CourseVersion`, `CurriculumVersion`, `CurriculumRule`, `CourseOffering`, `RegistrationAttempt`, `ResultRevision`, Exemption and Substitution.

Proposed additions: `AcademicStatusHistory`, `LeaveOfAbsence`, `GradingPolicy` and `GradeBand`. Split models into a package only when size justifies it.

Retained operations: `register_attempt(*, actor, student_id, offering_id, source_reference)`, `append_result_revision(*, actor, attempt_id, score, source_ref, reason)`, `explain_outstanding_requirements(*, actor, student_id, period_id)`.

Queries choose exact offerings, applicable curricula and selected accepted result revisions before calculations.

Academics module checkpoints

One course in two periods has separate offerings. Historical title/units remain tied to the correct `CourseVersion`.

An import retry preserves one source attempt. A genuine repeat survives. Corrections retain previous revisions and select one effective accepted revision per attempt.

An unmet requirement differs from an unknown requirement. Exemptions and substitutions need authority and applicable scope.

Leave and level require evidence. Approved academic rules determine GPA, progression and outstanding requirements. Calculations must report policy and completeness.

`app/imports/` files

models.py: `ImportBatch`, `StagingRow` and proposed `ImportConflict`. adapters.py: source readers/header mapping. validation.py: typed row errors.

services.py: `stage_batch(*, actor, file, source_system, period_id)`, `review_batch(*, actor, batch_id)`, `accept_batch(*, actor, batch_id, row_ids, idempotency_key)`.

queries.py: permitted previews/conflicts. forms.py: upload, source and explicit period. views.py: upload, preview, review and accept.

Checkpoint: preview never publishes accepted results. Acceptance rechecks authority and conflicts, calls academic services, and is safe under retries and concurrent requests.

app/accounts/ files

models.py: selected staff extension and scope records. services.py: authorised role/scope assignment and revocation. queries.py: allowed populations.

Proposed policies.py: `require_permission(actor, action, target)`, `permitted_student_scope(actor)`. Retain `can_export_records(user)` when used.

`forms.py/views.py/urls.py`: staff login and controlled maintenance. Keep role assignment out of public student forms.

Checkpoint: authenticated users still need action, row and field permission. Direct URL access and exports obey the same restrictions. Revocation takes effect on subsequent protected operations.

app/reporting/ files

models.py: ExportJob and, if approved here, StudentDocument. queries.py: permitted selections. writers.py: CSV/XLSX formatting of already permitted rows.

services.py: request_export(*, actor, selection), generate_export(export_job_id), authorise_download(*, actor, export_job_id).

views.py/urls.py: request, status and private download. Optional tasks.py delegates durable jobs after transaction commit.

Checkpoint: prohibited fields stay absent. Recheck access at generation/download, handle expiry, protect identifiers from spreadsheet conversion and agree formula-injection handling.

app/audit/ and app/identity/

audit/models.py: AuditEvent. audit/services.py: record_event(*, actor, action, target_type, target_id, reason, correlation_id). queries.py: authorised audit inspection.

Audit checkpoints: authenticated actor context, stable action codes, required transaction behaviour, no ordinary staff update/delete, no secret payload logging.

identity/models.py: device registry, credential mappings, requests and minimal events. api/ validates JSON and authenticates callers. services/requests.py, outcomes.py and resolution.py implement the workflow.

Identity checkpoints: active device, freshness, repeat-safe delivery, scoped mapping, minimal reply and unavailable failure. See the separate presentation.

UI and configuration writers

`app/templates/base.html`: shared layout. `students/create.html` and detail.html: canonical form fields and permitted context. `app/static/css/site.css` and `js/student-form.js`: presentation and small interactions.

HTML name="`full_name`" must match the form field. Browser validation helps users. Server forms/services enforce the rule.

`app/config/settings/`: environment settings. `config/urls.py`: include module routes. manage.py, asgi.py and wsgi.py: generated entry points after bootstrap.

Checkpoint: readable validation, CSRF for session writes, no credentials in frontend code, no hidden-field trust and no public links to private files.

PostgreSQL contributors

Review Django models and generated migrations as the shared schema source. Inspect SQL with sqlmigrate after bootstrap. Do not maintain a second conflicting hand-created schema.

Use UUID PKs, typed FKs, NOT NULL, reviewed unique keys and CHECK constraints. Use numeric for score precision and timestamptz for event instants.

Proposed checks: valid score range, ordered dates, positive credit units, unique period labels, unique revision numbers within an attempt and reviewed external-ID uniqueness.

`TextChoices` and Python validators alone do not guarantee every SQL writer obeys controlled states. Add database checks for critical finite state sets.

PostgreSQL indexes and concurrency

Candidate indexes follow lookups: external source/identifier, attempt student/offering, offering period/version, result attempt/review-state and active device/credential scope.

Unique constraints already create supporting unique indexes. Avoid redundant indexes. Measure query plans against an agreed synthetic workload.

A Python `exists()` check is insufficient under simultaneous inserts. Enforce the reviewed key, use transactions and handle the specific expected uniqueness conflict.

Lock the request/attempt row when consuming challenges or choosing one accepted revision. Consider a reviewed conditional unique index for one current record.

PostgreSQL completion checkpoints

Fresh database migrations succeed. Another contributor can apply the same migration history. Existing fictional rows survive staged required-field changes.

Database-level duplicate and range violations fail even when forms are bypassed. Relationship deletions follow explicit PROTECT/CASCADE choices.

Transactions roll back multi-row failure. Concurrent acceptance creates one logical result. Restore tests cover database and private file references.

Use bound SQL parameters. Development, tests and production need separate credentials. Migration/provisioning privileges differ from application runtime privileges.

Names for future code

Kind	Convention and example
Class	`PascalCase`: `ResultRevision`, `CsvSourceAdapter`
Function	`snake_case` verb: `append_result_revision`, `resolve_external_identity`
Variable/JSON field	`snake_case` noun: `offering_id`, score, `period_reference`
Boolean	Question-like: `is_active`, `is_complete`, `can_export`
Constant	`UPPER_SNAKE_CASE`: `MAX_UPLOAD_BYTES`, `IDENTITY_REQUEST_TTL_SECONDS`
Diagnostic code	Stable code: `score_missing`, `mapping_revoked`
Audit action	Dotted code: student.created, import.accepted

Names for future code (continued)

Use units in names: `timeout_seconds`, `file_size_bytes`. Never use data1, `student_number` for several different identifiers or an unexplained status.

One naming rule across layers

Use model relationship student and Django's generated `student_id` key attribute. `Student.student_id` itself is the UUID PK. student.pk refers to that same key.

Do not append _id to a `ForeignKey` field name: naming it `student_id` would produce confusing `student_id_id` behaviour.

API and import adapters may have a different representation, but shared meaning and explicit conversion must survive. score travels as text, Decimal, numeric and an agreed JSON decimal string.

New public names require a dictionary entry and caller review. Temporary local variables can vary without creating a second contract.

Shared handoff checklist

Each work card states owner, file, callable/class name, input types, output type, failure meanings, permissions and dependencies.

Provide one accepted fixture, one missing-value fixture, one denied case and one retry/conflict case where applicable.

Update `docs/data-dictionary.md`, migrations and contract examples together when a shared field changes. Record proposed/accepted status and reviewer.

Follow the repository's review policy. Documentation does not imply institutional approval. Use synthetic examples and keep real student records out of Git.

First connected build

1. Review the field dictionary and unresolved policy decisions.
2. Bootstrap Django/PostgreSQL and document reproducible setup.
3. Create one permitted student/profile with audit and a denied case.
4. Add one period, versioned course, offering and registration.
5. Stage and accept a fictional result twice without duplication.
6. Generate one permitted semester report and calculate under a fictional policy.
7. Run the simulated identity exchange separately, then connect staff lookup.

Each step ends with evidence and a clear handoff before expanding scope.

Decisions before production implementation

Decision	Owner
Source identifiers, authoritative corrections, field ownership	Student/data reviewer
Level/status/leave evidence, session meaning and attempts	Academic Data Steward
Grade scale, precision, repeats and rounding	Academic authority
Staff roles, cohorts and permitted fields	Department/access reviewer
Device authentication, matching scope and template location	API/firmware/biometric reviewers
File categories, retention, hosting and recovery	Operations/institution

Unsettled decisions can use labelled fictional fixtures. They must not silently become official policy.

Reference map

Project references: supplied 92-page oau-software-architecture-walkthrough.pdf, oau-eee-developer-handbook.md, current README.md and CONTRIBUTING.md.

Mechanics reference: https://docs.djangoproject.com/en/5.2/ref/models/fields/ for field and missing-value conventions.

Mechanics reference: https://www.postgresql.org/docs/current/datatype-numeric.html for exact numeric storage.

These references explain architecture and tool behaviour. New dictionary fields, lengths, model additions and function interfaces in this guide remain proposals for review.